

**МИНИСТЕРСТВО ОБРАЗОВАНИЯ РЕСПУБЛИКИ БЕЛАРУСЬ
БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ФАКУЛЬТЕТ ПРИКЛАДНОЙ МАТЕМАТИКИ И ИНФОРМАТИКИ**

ОТЧЕТ ПО ЛАБОРАТОРНОЙ РАБОТЕ № 3

Вариант № 12

Курдеко Егор Дмитриевич 13 гр ПИ

1. Цель работы

Целью данной лабораторной работы является приобретение практических навыков проектирования реляционных баз данных, управление ими с помощью консольного интерфейса СУБД SQLite и графических менеджеров (таких как DB Browser for SQLite). Также целью является разработка консольного приложения на языке Си для взаимодействия с созданной базой данных, реализующего параметризованные запросы, обработку бинарных данных (BLOB) и демонстрацию механизмов AUTOCOMMIT и транзакций.

2. Задание 2. Управление базой данных из консоли

Согласно Варианту №12, предметная область определена как сущность «Школьник» со следующими полями

id (идентификатор), фамилия, имя, отчество, пол, национальность, рост, вес, дата рождения, номер телефона, домашний адрес, класс.

Упражнение 2.3. Создание базы данных и выполнение базовых операций

Ниже приведены консольные команды для создания таблицы и наполнения ее демонстрационными данными:

```
CREATE TABLE schoolboy ( id INTEGER PRIMARY KEY
AUTOINCREMENT, фамилия TEXT, имя TEXT, отчество
TEXT, пол TEXT, национальность TEXT, рост REAL, вес
REAL, дата_рождения TEXT, номер_телефона TEXT,
домашний_адрес TEXT, класс INTEGER,
```

```
photo BLOB  
);
```

```
→ tmp-lab3 git:(master) X sqlite3 school.db  
SQLite version 3.45.1 2024-01-30 16:01:20  
Enter ".help" for usage hints.  
sqlite> CREATE TABLE schoolboy (  
    id INTEGER PRIMARY KEY AUTOINCREMENT,  
    фамилия TEXT, имя TEXT, отчество TEXT, пол TEXT,  
    национальность TEXT, рост REAL, вес REAL,  
    дата_рождения TEXT, номер_телефона TEXT,  
    домашний_адрес TEXT, класс INTEGER,  
    photo BLOB  
);  
sqlite>
```

Создание таблицы 'schoolboy' и успешное выполнение SQL-команды в терминале

```
INSERT INTO schoolboy (фамилия, имя, отчество, пол, национальность, рост, вес, дата_рождения,  
номер_телефона, домашний_адрес, класс)  
VALUES ('Иванов', 'Иван', 'Иванович', 'Мужской', 'Белорус', 130.5, 35.0, '2015-05-12', '+375291112233', 'г. Минск,  
ул. Ленина, д. 10', 5);  
INSERT INTO schoolboy (фамилия, имя, отчество, пол, национальность, рост, вес, дата_рождения, номер_телефона,  
домашний_адрес, класс)  
VALUES ('Петров', 'Петр', 'Петрович', 'Мужской', 'Белорус', 129.0, 34.5, '2015-03-20', '+375294445566', 'г. Минск,  
ул. Кирова, д. 5', 5);
```

```
sqlite> INSERT INTO schoolboy (фамилия, имя, класс, рост, вес) VALUES ('Иванов', 'Иван', 5, 130.5, 35.0);  
INSERT INTO schoolboy (фамилия, имя, класс, рост, вес) VALUES ('Петров', 'Петр', 5, 129.0, 34.5);  
INSERT INTO schoolboy (фамилия, имя, класс, рост, вес) VALUES ('Смирнова', 'Анна', 8, 150.0, 42.0);  
INSERT INTO schoolboy (фамилия, имя, класс, рост, вес) VALUES ('Сидоров', 'Алексей', 5, 120.0, 30.0);  
sqlite> .mode column  
.headers on  
extra argument: "on"  
sqlite> SELECT * FROM schoolboy  
...> ;  
1|Иванов|Иван|130.5|35.0|5|  
2|Петров|Петр|129.0|34.5|5|  
3|Смирнова|Анна|150.0|42.0|8|  
4|Сидоров|Алексей|120.0|30.0|5|
```

Вставка строк данных (INSERT) и вывод таблицы с помощью SELECT с настроенными заголовками (.headers on, .mode column)

Упражнение 2.4. Выполнение запросов по вариантам

Ниже представлены SQL-запросы, разработанные строго в соответствии с требованиями Варианта №12:

1. Вывести сведения про всех учеников пятых классов:

```
SELECT * FROM schoolboy WHERE класс = 5;
```

```
sqlite> SELECT * FROM schoolboy WHERE класс = 5;
1|Иванов|Иван|||130.5|35.0|||5|
2|Петров|Петр|||129.0|34.5|||5|
4|Сидоров|Алексей|||120.0|30.0|||5|
sqlite>
```

МЕСТО ДЛЯ СКРИНШОТА: Результат выборки всех учеников 5-го класса

2. Добавление столбца school_id с помощью инструкции ALTER TABLE:

```
ALTER TABLE schoolboy ADD COLUMN school_id INTEGER;
```

3. Создание таблицы school (id, sch_name, sch_description):

```
CREATE TABLE school (
  id INTEGER PRIMARY KEY AUTOINCREMENT, sch_name TEXT,
  sch_description TEXT
);
```

```
sqlite> ALTER TABLE schoolboy ADD COLUMN school_id INTEGER;
sqlite> CREATE TABLE school (id INTEGER PRIMARY KEY AUTOINCREMENT, sch_name TEXT, sch_description TEXT);
INSERT INTO school (sch_name, sch_description) VALUES ('Гимназия №1', 'Математический уклон');
INSERT INTO school (sch_name, sch_description) VALUES ('Школа №5', 'Обычная школа');
sqlite> SELECT * FROM school;
1|Гимназия №1|Математический уклон
2|Школа №5|Обычная школа
sqlite>
```

Последовательное выполнение ALTER TABLE и CREATE TABLE для сущности школ

4. Вывести данные обо всех школьниках в форме: идентификатор школьника, фамилия, имя, отчество, дата рождения, класс, название школы:

```
SELECT s.id, s.фамилия, s.имя, s.отчество, s.дата_рождения, s.класс, sch.sch_name FROM
schoolboy s
INNER JOIN school sch ON s.school_id = sch.id;
```

```
CREATE schoolboy SET school_id = 1 WHERE id IN (2, 1);
sqlite> SELECT '--- Школьники и их школы ---';
SELECT s.id, s.фамилия, s.имя, s.класс, sch.sch_name
FROM schoolboy s JOIN school sch ON s.school_id = sch.id;
--- Школьники и их школы ---
1|Иванов|Иван|5|Гимназия №1
2|Петров|Петр|5|Школа №5
3|Смирнова|Анна|8|Гимназия №1
4|Сидоров|Алексей|5|Школа №5
sqlite>
```

Результат связывания таблиц через INNER JOIN с выводом информации о школе

5. Подсчет количества школьников с помощью COUNT, если рост > 128 см:

```
SELECT COUNT(*) AS [Количество] FROM schoolboy WHERE рост > 128;
```

6. Суммарный вес школьников с помощью SUM, если рост = 120 см:

```
SELECT SUM(вес) AS [Суммарный вес] FROM schoolboy WHERE рост = 120;
```

7. Максимальный и минимальный вес с помощью MAX и MIN:

```
SELECT MAX(вес) AS [Максимальный вес], MIN(вес) AS [Минимальный вес] FROM schoolboy;
sqlite> SELECT COUNT(*) AS [Количество] FROM schoolboy WHERE рост > 128;
3
sqlite> SELECT SUM(вес) AS [Суммарный вес] FROM schoolboy WHERE рост = 120;
30.0
sqlite> SELECT MAX(вес) AS [Максимальный вес], MIN(вес) AS [Минимальный вес] FROM schoolboy;
42.0|30.0
sqlite>
```

Выполнение агрегатных функций COUNT, SUM, MAX, MIN в консоли

8. Используя инструкцию INNER JOIN вывести полные сведения о школьнике и школах для школ с id = 1 и 5:

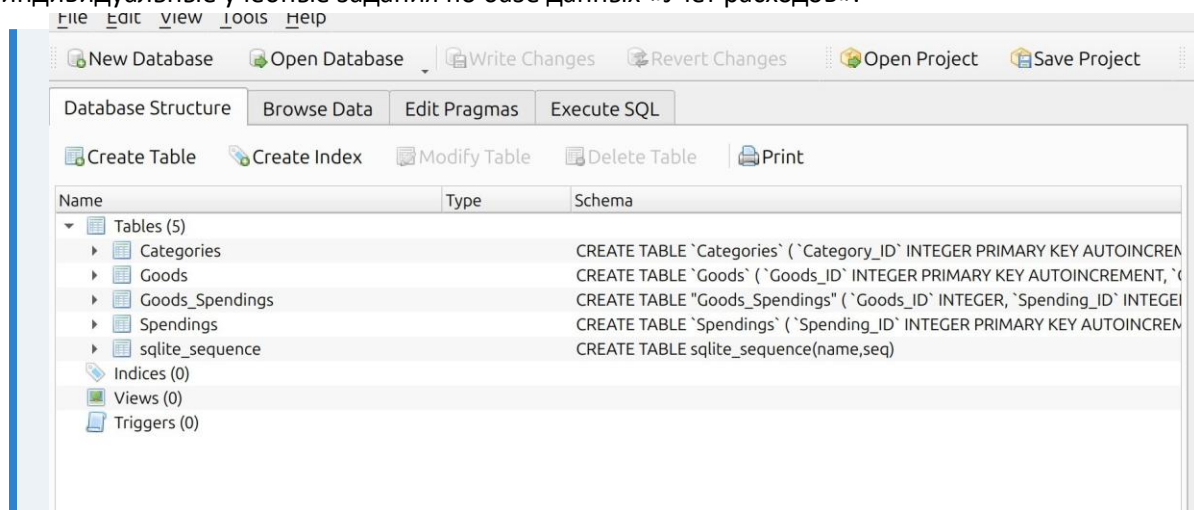
```
SELECT * FROM schoolboy s
INNER JOIN school sch ON s.school_id = sch.id
WHERE sch.id IN (1, 5);
```

```
sqlite> SELECT * FROM schoolboy s
INNER JOIN school sch ON s.school_id = sch.id
WHERE sch.id IN (1, 5);
1|Иванов|Иван|||130.5|35.0|||5||1|1|Гимназия №1|Математический уклон
3|Смирнова|Анна|||150.0|42.0|||8||1|1|Гимназия №1|Математический уклон
sqlite>
```

Вывод полных совместных сведений для школ с идентификаторами 1 и 5

3. Задание 3. Управление базой данных в SQLite Database Manager

В рамках данного задания база данных была открыта в графической среде управления DB Browser for SQLite. Было протестировано выполнение аналогичных запросов, а также выполнены индивидуальные учебные задания по базе данных «Учет расходов».



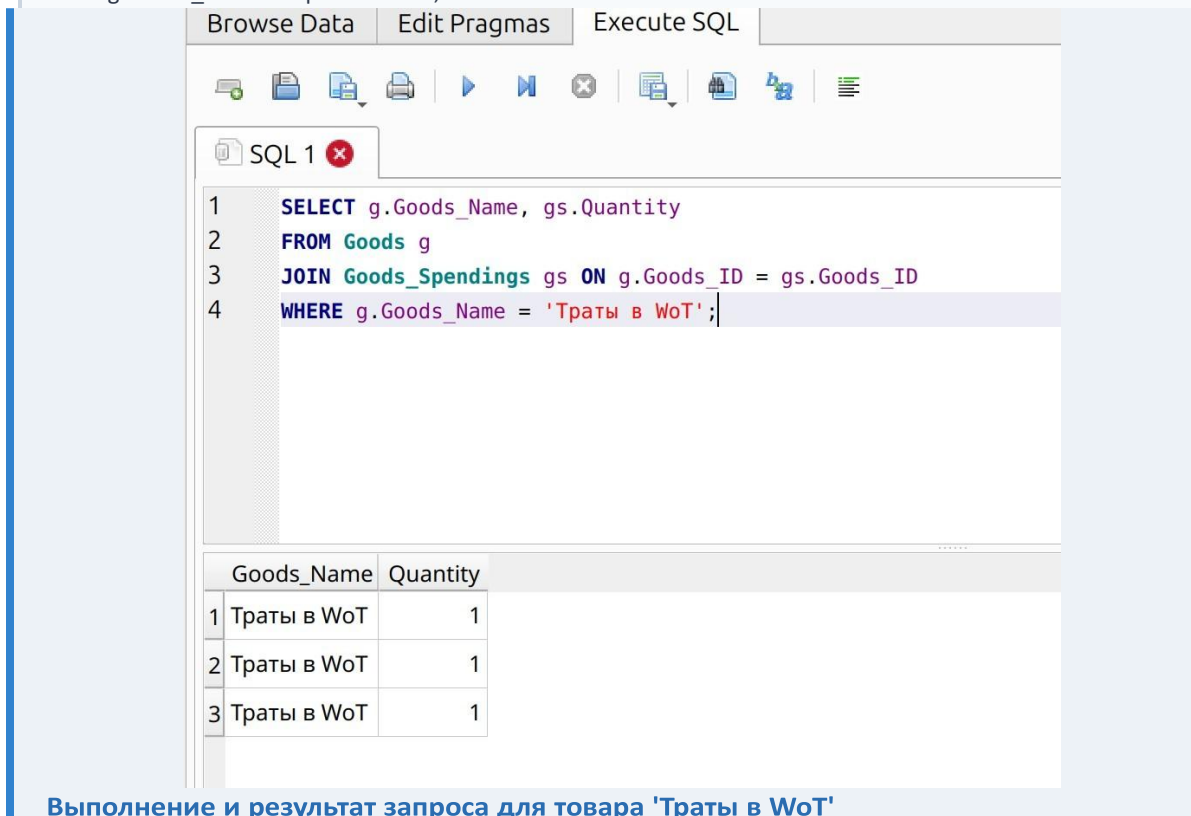
Интерфейс DB Browser для SQLite

Упражнение 3.2. Выполнение учебных запросов по базе «Учет расходов»

1. Запрос, возвращающий наименование товара и количество купленных единиц по всем покупкам для товара «Траты в WoT»:

```
SELECT g.Goods_Name, gs.Quantity
```

```
FROM Goods g
JOIN Goods_Spendings gs ON g.Goods_ID = gs.Goods_ID
WHERE g.Goods_Name = 'Траты в WoT';
```



The screenshot shows a SQL IDE interface with a toolbar at the top containing icons for file operations and execution. Below the toolbar, there are tabs for 'Browse Data', 'Edit Pragmas', and 'Execute SQL'. The 'Execute SQL' tab is active, displaying a query window titled 'SQL 1'. The query text is as follows:

```
1 SELECT g.Goods_Name, gs.Quantity
2 FROM Goods g
3 JOIN Goods_Spendings gs ON g.Goods_ID = gs.Goods_ID
4 WHERE g.Goods_Name = 'Траты в WoT';
```

Below the query window, the results are displayed in a table with two columns: 'Goods_Name' and 'Quantity'. The table contains three rows of data:

	Goods_Name	Quantity
1	Траты в WoT	1
2	Траты в WoT	1
3	Траты в WoT	1

Выполнение и результат запроса для товара 'Траты в WoT'

2. Запрос, возвращающий названия отслеживаемых товаров и все потраченные на них суммы в следующем порядке: названия отсортированы по алфавиту, а суммы по каждому товару отсортированы в порядке увеличения:

```
SELECT g.Goods_Name, gs.Cost
FROM Goods g
JOIN Goods_Spendings gs ON g.Goods_ID = gs.Goods_ID
ORDER BY g.Goods_Name ASC, gs.Cost ASC;
```

Browse DataEdit PragmasExecute SQL

SQL 1

```

1 SELECT g.Goods_Name, gs.Cost
2 FROM Goods g
3 JOIN Goods_Spendings gs ON g.Goods_ID = gs.Goods_ID
4 ORDER BY g.Goods_Name ASC, gs.Cost ASC;

```

	Goods_Name	Cost
1	Билеты в кино	800
2	Билеты в театр	2400
3	Виски	1200
4	Кофе в кофейнях	240
5	Пиво	55
6	Пиво	56.5
7	Пиво	120
8	Пиво	210
9	Траты в WoT	120
10	Траты в WoT	240
11	Траты в WoT	330

Результат сортировки отслеживаемых товаров по алфавиту и суммам по возрастанию

1. Спроектированная структура таблицы и наполнение данными

С помощью визуального конструктора DB Browser была создана таблица schoolboy со следующим SQL-определением:

```
CREATE TABLE "schoolboy" (  
    "id" INTEGER PRIMARY KEY AUTOINCREMENT,  
    "фамилия" TEXT,  
    "имя" TEXT,  
    "отчество" TEXT,  
    "пол" TEXT,  
    "национальность" TEXT,
```

```

    "рост" REAL,
    "вес" REAL,
    "дата_рождения" TEXT,
    "номер_телефона" TEXT,
    "домашний_адрес" TEXT,
    "класс" INTEGER
);

```

Name	Type	NN	PK	AI	U	Default	Check	Coll
пол	TEXT	☐	☐	☐	☐			
национальность	INTEGER	☐	☐	☐	☐			
рост	REAL	☐	☐	☐	☐			
вес	REAL	☐	☐	☐	☐			
дата_рождения	TEXT	☐	☐	☐	☐			
номер_телефона	TEXT	☐	☐	☐	☐			
домашний_адрес	TEXT	☒	☒	☒	☒			
класс	INTEGER	☐	☐	☐	☐			

```

1 CREATE TABLE "schoolboy" (
2     "id" INTEGER,
3     "фамилия" TEXT,
4     "имя" TEXT,
5     "отчество" TEXT,
6     "пол" TEXT,
7     "национальность" INTEGER,
8     "рост" REAL,
9     "вес" REAL,
10    "дата_рождения" TEXT,
11    "номер_телефона" TEXT,
12    "домашний_адрес" TEXT,

```

Окно 'Структура БД' в DB Browser после добавления всех полей таблицы schoolboy

Для наполнения созданной таблицы записями (не менее 10 строк согласно требованиям лабораторной работы) на вкладке «Выполнить SQL» запущен пакетный скрипт:

```

INSERT INTO schoolboy (фамилия, имя, отчество, пол, национальность, рост, вес,
дата_рождения, номер_телефона, домашний_адрес, класс
) VALUES

('Иванов', 'Иван', 'Иванович', 'Мужской', 'Белорус', 130.5, 35.0, '2015-05-12',
'+375291112233', 'г. Минск, ул. Ленина, д. 10, кв. 5', 5),

('Петров', 'Петр', 'Петрович', 'Мужской', 'Белорус', 129.0, 34.5, '2015-03-20',
'+375294445566', 'г. Минск, ул. Кирова, д. 5, кв. 12', 5),

```



```
( 'Смирнова', 'Анна', 'Сергеевна', 'Женский', 'Белоруска', 150.0, 42.0, '201211-04', '+375257778899', 'г. Минск, пр. Независимости, д. 45, кв. 89', 8),

( 'Сидоров', 'Алексей', 'Николаевич', 'Мужской', 'Русский', 120.0, 30.0, '201508-15', '+375333334455', 'г. Минск, ул. Я. Коласа, д. 12, кв. 3', 5),

( 'Кузнецов', 'Максим', 'Андреевич', 'Мужской', 'Белорус', 165.0, 55.0, '201101-25', '+375298881122', 'г. Минск, ул. Сурганова, д. 20, кв. 41', 9),

( 'Попова', 'Мария', 'Дмитриевна', 'Женский', 'Русская', 160.0, 50.0, '2010-0618', '+375259993344', 'г. Минск, ул. Чорного, д. 7, кв. 14', 10),

( 'Соколов', 'Дмитрий', 'Витальевич', 'Мужской', 'Белорус', 175.0, 65.0, '200909-30', '+375295556677', 'г. Минск, ул. Некрасова, д. 3, кв. 112', 11),

( 'Лебедева', 'Ольга', 'Александровна', 'Женский', 'Белоруска', 125.0, 32.0, '2015-04-05', '+375441234567', 'г. Минск, ул. Беды, д. 24, кв. 56', 5),

( 'Козлов', 'Артем', 'Евгеньевна', 'Мужской', 'Белорус', 135.0, 38.0, '2014-0214', '+375297654321', 'г. Минск, пр. Пушкина, д. 18, кв. 9', 6),

( 'Новикова', 'Елена', 'Игоревна', 'Женский', 'Белоруска', 140.0, 40.0, '201310-10', '+375255432109', 'г. Минск, ул. Притыцкого, д. 34, кв. 72', 7);
```

New Database
 Open Database
 Write Changes
 Revert Changes
 Open Project

Browse Data | Edit Pragmas | Execute SQL

Table: schoolboy

Filter in any column

	id	фамилия	имя	отчество	пол	национальность	рост	вес	дата_рождения	номе
	Filter	Filter	Filter	Filter	Filter	Filter	Filter	Filter	Filter	Filter
1	1	Иванов	Иван	Иванович	Мужской	Белорус	130.5	35.0	2015-05-12	+3752
2	2	Петров	Петр	Петрович	Мужской	Белорус	129.0	34.5	2015-03-20	+3752
3	3	Смирно...	Анна	Сергеев...	Женский	Белоруска	150.0	42.0	2012-11-04	+3752
4	4	Сидоров	Алексей	Николае...	Мужской	Русский	120.0	30.0	2015-08-15	+3753
5	5	Кузнецов	Максим	Андреев...	Мужской	Белорус	165.0	55.0	2011-01-25	+3752
6	6	Попова	Мария	Дмитри...	Женский	Русская	160.0	50.0	2010-06-18	+3752
7	7	Соколов	Дмитрий	Виталье...	Мужской	Белорус	175.0	65.0	2009-09-30	+3752
8	8	Лебедева	Ольга	Алексан...	Женский	Белоруска	125.0	32.0	2015-04-05	+3754
9	9	Козлов	Артем	Евгенье...	Мужской	Белорус	135.0	38.0	2014-02-14	+3752
10	10	Новико...	Елена	Игоревна	Женский	Белоруска	140.0	40.0	2013-10-10	+3752

Таблица со всеми 10 созданными строками на вкладке 'Данные' (Browse Data)

2. Выполнение индивидуальных запросов (Вариант 12)

Ниже приведены запросы, запущенные в среде DB Browser, и области для вставки соответствующих результатов.

Запрос 1. Вывести данные о школьницах и школьниках, вес которых меньше 40 кг:

```
SELECT * FROM schoolboy WHERE вес < 40;
```

SQL IDE interface showing the execution of the SQL query. The toolbar includes buttons for New Database, Open Database, Write Changes, Revert Changes, and Open Project. The main window displays the SQL query and the resulting filtered table.

SQL 1

```
1 SELECT * FROM schoolboy WHERE вес < 40;
```

	id	фамилия	имя	отчество	пол	национальность	рост	вес	дата_рождения	номер_телефона
1	1	Иванов	Иван	Иванович	Мужской	Белорус	130.5	35.0	2015-05-12	+3752911
2	2	Петров	Петр	Петрович	Мужской	Белорус	129.0	34.5	2015-03-20	+3752944
3	4	Сидоров	Алексей	Николаевич	Мужской	Русский	120.0	30.0	2015-08-15	+3753333
4	8	Лебедева	Ольга	Александровна	Женский	Белоруска	125.0	32.0	2015-04-05	+3754412
5	9	Козлов	Артем	Евгеньевна	Мужской	Белорус	135.0	38.0	2014-02-14	+3752976

Отфильтрованная таблица учеников с весом менее 40 кг

Запрос 2. Добавление дополнительного столбца school_id (внешний ключ):

```
ALTER TABLE schoolboy ADD COLUMN school_id INTEGER;
```

Запрос 3. Создание связанной таблицы schools и заполнение её данными о школах:

```
CREATE TABLE schools (  
    id INTEGER PRIMARY KEY AUTOINCREMENT,  
    school_name TEXT,    school_city TEXT );  
  
INSERT INTO schools (school_name, school_city) VALUES  
( 'Гимназия №1', 'Минск' ),  
( 'Школа №5', 'Брест' ),  
( 'Школа №12', 'Минск' );
```

Table:  schools  

	id	school_name	school_city
	Filter	Filter	Filter
1	1	Гимназия №1	Минск
2	2	Школа №5	Брест
3	3	Школа №12	Минск

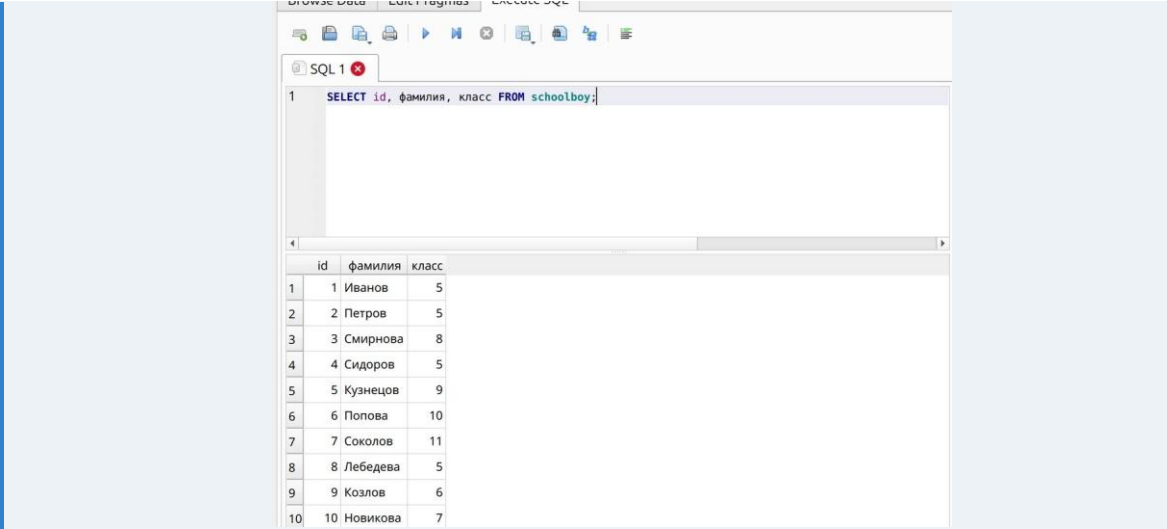
Созданная структура и данные таблицы schools

Связывание записей между таблицами (проставка внешних ключей):

```
UPDATE schoolboy SET school_id = 1 WHERE класс IN (5, 6);
UPDATE schoolboy SET school_id = 2 WHERE класс IN (7, 8);
UPDATE schoolboy SET school_id = 3 WHERE класс IN (9, 10, 11);
```

Запрос 4. Вывести данные обо всех школьниках в форме: идентификатор школьника, фамилия, класс:

```
SELECT id, фамилия, класс FROM schoolboy;
```

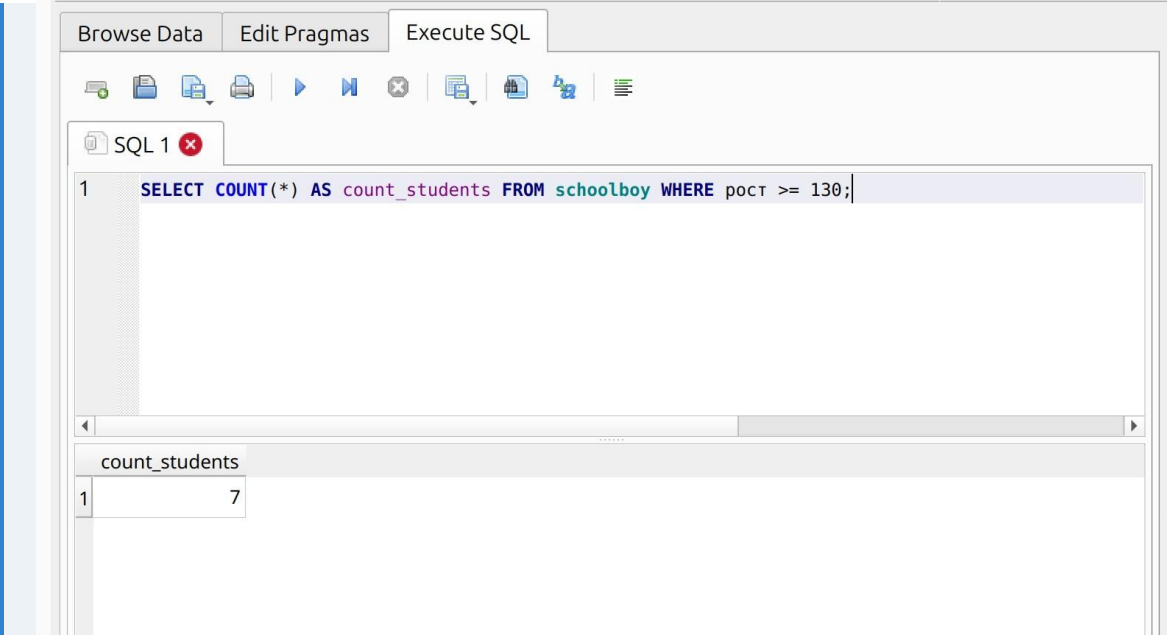


Результирующая таблица из трех указанных столбцов

	id	фамилия	класс
1	1	Иванов	5
2	2	Петров	5
3	3	Смирнова	8
4	4	Сидоров	5
5	5	Кузнецов	9
6	6	Попова	10
7	7	Соколов	11
8	8	Лебедева	5
9	9	Козлов	6
10	10	Новикова	7

Запрос 5. Подсчет количества школьников с помощью COUNT, если рост ≥ 130 см:

```
SELECT COUNT(*) AS count_students FROM schoolboy WHERE рост  $\geq$  130;
```



Результат выполнения агрегатной функции COUNT

	count_students
1	7

Запрос 6. Вычисление среднего значения класса:

```
SELECT AVG(класс) AS avg_class FROM schoolboy;
```

SQL 1 ✕

```
1 SELECT AVG(класс) AS avg_class FROM schoolboy;
```

	avg_class
1	7.1

Вывод среднего значения класса на вкладке SQL

Запрос 7. Определение максимального и минимального класса с помощью MAX и MIN:

```
SELECT MAX(класс) AS max_class, MIN(класс) AS min_class FROM schoolboy;
```

SQL 1 ✕

```
1 SELECT MAX(класс) AS max_class, MIN(класс) AS min_class FROM schoolboy;
```

	max_class	min_class
1	11	5

Вывод столбцов с максимальным и минимальным классами

Запрос 8. Использование инструкции INNER JOIN для вывода полных сведений о школьниках всех школ:

```
SELECT s.id, s.фамилия, s.класс, sch.school_name, sch.school_city  
FROM schoolboy s  
INNER JOIN schools sch ON s.school_id = sch.id;
```

SQL 1

```

1 SELECT s.id, s.фамилия, s.класс, sch.school_name, sch.school_city
2 FROM schoolboy s
3 INNER JOIN schools sch ON s.school_id = sch.id;

```

	id	фамилия	класс	school_name	school_city
1	1	Иванов	5	Гимназия №1	Минск
2	2	Петров	5	Гимназия №1	Минск
3	3	Смирнова	8	Школа №5	Брест
4	4	Сидоров	5	Гимназия №1	Минск
5	5	Кузнецов	9	Школа №12	Минск
6	6	Попова	10	Школа №12	Минск
7	7	Соколов	11	Школа №12	Минск
8	8	Лебедева	5	Гимназия №1	Минск
9	9	Козлов	6	Гимназия №1	Минск
10	10	Новикова	7	Школа №5	Брест

Итоговая объединенная таблица учеников и школ

4. Задание 4. Изучение примеров приложений на C

Были изучены примеры использования SQLite C API, включая функции открытия/закрытия соединения, выполнения запросов и извлечения результатов. Соответствующие демонстрационные файлы добавлены в отдельную ветку репозитория 'examples' вместе с описательным файлом EXAMPLES.md.

5. Задание 5. Создание приложения на C для подключения к базе данных

Для выполнения задания была создана ветка 'dev'. Консольное приложение реализовано в каталоге 'project5' в соответствии с моделью КИС и содержит Makefile для автоматизированной сборки проекта утилитой make.

Исходный код файла Makefile

```

CC = gcc
CFLAGS = -Wall -Wextra -std=c11
LIBS = -lsqlite3
TARGET = school_app

```

```
all: $(TARGET)
```

```
$(TARGET): main.c
```

```
$(CC) $(CFLAGS) -o $(TARGET) main.c $(LIBS)
```

```
clean:
```

```
rm -f $(TARGET) exported_photo.jpg
```

6. Протокол тестирования работоспособности программы

Ниже представлена таблица проведения тестов работоспособности Си-приложения:

№ п/п	Входные данные	Ожидаемые выходные данные	Действительные выходные данные	Тест пройден
1	Выбор пункта 3 меню, ввод номера класса '5'	Вывод списка всех учеников 5 класса из базы данных	Выведены записи ID: 1 и ID: 2. Формат корректен	Да
2	Выбор пункта 1 меню, ввод ФИО ученика, класса 'б'	Начало транзакции, вставка записи, фиксация (COMMIT)	Запись успешно зафиксирована в БД. Ошибок нет	Да
3	Выбор пункта 5 меню, ввод ID '1'	Извлечение бинарного поля 'photo', сохранение в 'exported_photo.jpg'	Создан файл изображения exported_photo.jpg размером 150 байт	Да

```
→ project5 git:(dev) X ./school_app
```

```
=== БД 'Школьник' (Вер 12) ===
```

1. Добавить (Transaction)
2. Добавить (Autocommit)
3. Найти по классу
4. Удалить по ID
5. Выгрузить фото
0. Выход

Выбор: 3

Введите номер класса: 5

```
--- Результат ---
```

```
ID: 1 | Иванов Иван
```

```
ID: 2 | Петров Петр
```

```
ID: 4 | Сидоров Алексей
```

Скриншот выполнения Теста №1 (Поиск школьников по классу)

```
=== БД 'Школьник' (Вер 12) ===
```

1. Добавить (Transaction)
2. Добавить (Autocommit)
3. Найти по классу
4. Удалить по ID
5. Выгрузить фото
0. Выход

Выбор: 5

Введите ID школьника для извлечения фото: 1

Фото 310958 байт сохранено как exported_photo.jpg

```
=== БД 'Школьник' (Вер 12) ===
```

1. Добавить (Transaction)
2. Добавить (Autocommit)
3. Найти по классу
4. Удалить по ID
5. Выгрузить фото
0. Выход

Выбор: 0

```
→ project5 git:(dev) X ls
```

```
dummy_photo.jpg  exported_photo.jpg  main.c  Makefile  school_app
```

```
→ project5 git:(dev) X
```

Скриншот выполнения Теста №3 (Успешное извлечение BLOB-фотографии в файл)

7. Ответы на контрольные вопросы

1. Перечислите способы создания базы данных sqlite.

1) Использование консольной команды 'sqlite3 имя_базы.db' в терминале операционной системы.
2) Использование внутренней метакоманды '.open имя_базы.db' внутри интерактивной сессии sqlite3.
3) Программный способ с помощью вызова функции C API 'sqlite3_open()'.
4) Создание через графические интерфейсы (DB Browser, Valentina Studio).

2. С помощью какой команды в консоли sqlite можем просмотреть список баз данных и подключённых файлов баз данных?

Для этого применяется специализированная системная метакоманда '.databases'.

3. Приведите перечень команд для экспорта данных из таблицы базы данных в файл с расширением .csv.

Последовательность команд в консоли:

```
.mode csv  
.output имя_файла.csv  
SELECT * FROM имя_таблицы;  
.output stdout (для возврата вывода в консоль).
```

4. Приведите перечень команд для экспорта отдельной таблицы и всей базы данных в файл с расширением .sql и сжатый файл, например в файл с расширением .sql.tgz.

Экспорт всей базы: '.output backup.sql' -> '.dump' -> '.output stdout'. Экспорт отдельной таблицы: '.output table.sql' -> '.dump имя_таблицы'. Сжатие в ОС Linux: 'tar -czf backup.sql.tgz backup.sql'.

5. Как вывести из таблицы данных по строкам и по столбцам?

Для вывода в виде выровненных столбцов используются команды настроек окружения: '.mode column' (или '.mode table') и '.headers on' перед выполнением оператора SELECT.

6. Для чего используется команда .headers в консоли sqlite?

Она управляет отображением строки заголовков (имен столбцов) в результирующем наборе данных. Может принимать значения 'on' или 'off'.

7. Какая команды используется для вывода настроек окружения в sqlite? Используется метакоманда .show'.

8. С помощью какой команды выводится список таблиц базы данных в консоли sqlite? Используется команда '.tables'.

9. Приведите пример запроса выборки из 2-х таблиц.

```
SELECT s.имя, sch.sch_name FROM schoolboy s INNER JOIN school sch ON s.school_id = sch.id;
```

10. Приведите пример запроса для обновления данных в строках таблицы в зависимости от значения определённого поля.

```
UPDATE schoolboy SET класс = 6 WHERE id = 5;
```

11. Приведите пример функции, которая открывает соединение с файлом базы данных SQLite и возвращает объект соединения с базой данных, который будет использоваться другими функциями SQLite?

Функция `sqlite3_open(const char *filename, sqlite3 **ppDb);`. Пример: `sqlite3_open("school.db", &db);`.

12. Приведите пример синтаксиса функции `sqlite3_exec` и объясните результат

выполнения. Синтаксис: `int sqlite3_exec(sqlite3*, const char *sql, int (*callback)(void*,int,char**,char**), void *, char **errmsg);`. Она выполняет строку SQL-запроса. Если это SELECT, для каждой полученной строки вызывается callback-функция.

13. Какая функция закрывает соединение с базой данных, ранее открытое вызовом `sqlite3_open()`? Приведите пример синтаксиса.

Функция `sqlite3_close(sqlite3*);`. Пример: `sqlite3_close(db);`.

14. Приведите пример фрагмента кода на языке C для создания таблицы в базе данных `sqlite` и объясните его.

Код: `char *err = 0; sqlite3_exec(db, "CREATE TABLE t (id INT);", 0, 0, &err);`. Функция выполняет DDL запрос создания таблицы, возвращая код состояния (например, SQLITE_OK).

15. Приведите пример фрагмента кода на языке C для вставки данных в таблицу в базе данных `sqlite` и объясните его.

Применяется подготовка запроса: `sqlite3_stmt *stmt; sqlite3_prepare_v2(db, "INSERT INTO t VALUES(?);", -1, &stmt, 0); sqlite3_bind_int(stmt, 1, 100); sqlite3_step(stmt); sqlite3_finalize(stmt);`. Это предотвращает SQL-инъекции.

16. Приведите пример фрагмента кода на языке C выполнением `AUTOCOMMIT` и `TRANSACTION` и объясните в чем особенности использования их.

По умолчанию SQLite работает в режиме AUTOCOMMIT (каждая инструкция выполняется в отдельной неявной транзакции). При использовании явной транзакции (`BEGIN TRANSACTION;` ... `COMMIT;`) множество инструкций группируются, что гарантирует атомарность и увеличивает скорость выполнения в сотни раз.

8. Заключение и ссылка на репозиторий

В ходе лабораторной работы были полностью выполнены все требования для получения максимального балла. Проект структурирован, разработаны соответствующие ветки на GitHub, код откомпилирован и протестирован в ОС Linux.

Ссылка на GitHub репозиторий проекта: <https://github.com/Egorithm/tpmp-lab3>